100 Most Important DSA Questions

for Accenture 2026 On-Campus (2027 Batch) — Java Edition

With LeetCode references, worked examples, clean commented code, and teaching notes on the trickiest patterns.

How Accenture’s Coding Round Actually Works

(Based on recent 2025–2026 candidate reports)

- **Format:** 2 coding questions, 45 minutes total, no negative marking. Java/C/C++/Python/.NET all accepted.
- **Difficulty:** Mostly **Easy–Medium**. One question is usually easy, the other medium; hard questions are rare.
- **Topics that dominate: Arrays and Strings by far** (“leader element,” longest substring, array/matrix manipulation appear in almost every reported test), followed by basic **Matrices**, **Linked List**, **Stacks/Queues**, number-theory-style **Math** (Armstrong number, round number, prime check), and pseudocode/output-prediction questions (a separate MCQ round, not covered in this guide).
- **Rarely asked in the coding round:** Graphs, heavy Dynamic Programming, advanced tree algorithms, tries, segment trees — which is why this guide leans heavily on Arrays, Strings, Binary Search, and core linear data structures instead.
- **Scoring:** Test-case based, with partial credit. Aim to fully pass at least one question and pass as many hidden test cases as possible on the other — clean, working code beats clever-but-buggy code.
- Binary Search, Linked List, and Stack/Queue questions show up more in the **technical interview round** (especially for the Advanced ASE track) than in the timed 45-minute coding round — but they’re covered here in good depth since interviewers do ask them directly.

How to Use This Guide

- 100 questions grouped by topic, in roughly increasing difficulty within each group.
- Every entry follows the same structure: **Source** (LeetCode link + number where one exists, or an honest note when it's a classic GfG/interview problem instead) → **Problem statement** → **Worked example(s)** → **Approach** (with a deeper **Teaching note** for the genuinely tricky ones) → **Clean, commented Java code** → **Time/space complexity**.
- All solutions are written as **static methods** — drop them into a `Solution` class and call them from `main`, e.g.:

```java
public class Solution {
    public static void main(String[] args) {
        int[] arr = {2, 5, 1, 8, 3};
        System.out.println(findLargest(arr)); // call any method from this guide
    }
    // paste a method here
}
```

- **Practice priority for Accenture specifically:** Arrays → Strings → Matrices → Linked List → Stack/Queue → Binary Search → HashMap → Sorting → Recursion/Math.
- A quick note on the LeetCode links: many classic interview staples (e.g., "second largest element," "leaders in an array") are GeeksforGeeks-style problems that were never given their own LeetCode number. Rather than force a fake link, this guide says so plainly and points you to the closest real LeetCode relative where one exists, so you can practice the pattern either way.

Section 1: Arrays (Q1–Q25)

Q1. Find the Largest Element in an Array

Source: Not on LeetCode — classic foundational problem (GfG: *Largest Element in Array*). This exact "leader element" style question has been directly reported in Accenture's 2026 coding round.

Problem: Given an array of integers, find the maximum value in it.

Example

```
Input:  arr = [2, 5, 1, 8, 3]
Output: 8
Explanation: 8 is bigger than every other element in the array.
```

Approach: Assume the first element is the largest, then walk through the rest of the array once, updating your answer whenever you find something bigger.

```java
        static int findLargest(int[] arr) {
            int max = arr[0];               // start with the first element as a guess
            for (int x : arr) {
                if (x > max) max = x;       // update whenever we see something bigger
            }
            return max;
        }
```

Complexity: Time O(n), Space O(1)

Q2. Find the Second Largest Element in an Array

Source: Not on LeetCode as a standalone number — classic GfG problem. (Related: LeetCode 414 *Third Maximum Number* uses the same "track the top few" idea.)

Problem: Find the second-largest **distinct** value in an array, in a single pass.

Example

```
Input:  arr = [12, 35, 1, 10, 34, 1]
Output: 34
Explanation: 35 is the largest, 34 is the next distinct value below it.
```

Approach: Keep two trackers, `first` and `second`. Whenever a new element beats `first`, the old `first` slides down into `second`. Whenever an element beats `second` but not `first` (and isn't equal to `first`), it becomes the new `second`.

```java
        static int secondLargest(int[] arr) {
            int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
            for (int x : arr) {
                if (x > first) {
                    second = first;   // old largest demoted to second
                    first = x;
                } else if (x > second && x != first) {
                    second = x;
                }
            }
            return second;
        }
```

Complexity: Time O(n), Space O(1)

Q3. Reverse an Array In Place

Source: Not directly its own LeetCode number, but the identical two-pointer swap technique is used in LeetCode 344 *Reverse String*.

Problem: Reverse the order of elements in an array without using extra space.

Example

```
Input:  arr = [1, 2, 3, 4, 5]
Output: [5, 4, 3, 2, 1]
```

Approach: Two pointers start at opposite ends and swap inward until they cross.

```
        static void reverseArray(int[] arr) {
            int l = 0, r = arr.length - 1;
            while (l < r) {
                int temp = arr[l];
                arr[l] = arr[r];
                arr[r] = temp;
                l++;
                r--;
            }
        }
```

Complexity: Time O(n), Space O(1)

Q4. Rotate an Array Left by k Positions

Source: LeetCode 189 — Rotate Array (LC rotates *right*; rotating left by k is the mirror image, and the reversal trick below handles either direction).

Problem: Shift every element k places to the left, wrapping the first k elements around to the end.

Example

```
Input:  arr = [1, 2, 3, 4, 5, 6, 7], k = 2
Output: [3, 4, 5, 6, 7, 1, 2]
Explanation: The first two elements (1, 2) wrap around to the back.
```

Approach — the "reversal trick": Reverse the whole array, then reverse each of the two pieces separately. Reversing the whole thing puts everything in the right relative *group*, and the two smaller reversals fix the internal order of each group back to normal.

> **Teaching note:** This is a favorite Accenture-style trick because it needs zero extra array. Walk through `[1,2,3,4,5,6,7]`, k=2: reverse all → `[7,6,5,4,3,2,1]`; reverse first `n-k=5` → `[3,4,5,6,7,2,1]`; reverse last `k=2` → `[3,4,5,6,7,1,2]`. Done — and it's exactly the rotated array.

```
        static void rotateLeft(int[] arr, int k) {
            int n = arr.length;
            k %= n;                          // handle k > n
            reverse(arr, 0, n - 1);          // reverse everything
            reverse(arr, 0, n - k - 1);      // fix the first (n-k) part
            reverse(arr, n - k, n - 1);      // fix the last k part
        }
        static void reverse(int[] arr, int l, int r) {
            while (l < r) {
                int t = arr[l]; arr[l] = arr[r]; arr[r] = t;
                l++; r--;
            }
        }
```

Complexity: Time O(n), Space O(1)

Q5. Maximum Subarray Sum — Kadane's Algorithm

Source: LeetCode 53 — Maximum Subarray

Problem: Find the contiguous subarray (containing at least one number) with the largest sum, and return that sum.

Example

```
Input:  arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: The subarray [4, -1, 2, 1] has the largest sum = 6.
```

Approach: At every index, decide: is it better to *extend* the running subarray, or *start fresh* from here? Keep whichever is bigger.

> **Teaching note — why this works:** `curMax` answers "what's the best subarray sum that *ends exactly at index i*?" A negative running sum can only drag future sums down, so the moment `curMax` would become worse than starting over (`arr[i]` alone), we restart. `maxSoFar` just remembers the best `curMax` we've seen across all positions. This single insight — "a negative prefix is never worth carrying forward" — is what turns an O(n²) brute force (try every subarray) into O(n).

```
        static int maxSubArraySum(int[] arr) {
            int maxSoFar = arr[0];
            int curMax = arr[0];
            for (int i = 1; i < arr.length; i++) {
                // either extend the previous subarray, or start a new one at i
                curMax = Math.max(arr[i], curMax + arr[i]);
                maxSoFar = Math.max(maxSoFar, curMax);
            }
            return maxSoFar;
        }
```

Complexity: Time O(n), Space O(1)

Q6. Move All Zeroes to the End (Keeping Relative Order)

Source: LeetCode 283 — Move Zeroes

Problem: Move all 0s to the end of the array while keeping the relative order of the non-zero elements, in place.

Example

```
Input:  arr = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

Approach: Use a write-pointer `insertPos` that only advances when we place a non-zero value; afterward, fill the remaining tail with zeroes.

```
        static void moveZeroes(int[] arr) {
            int insertPos = 0;
            for (int x : arr) {
                if (x != 0) arr[insertPos++] = x;    // pack non-zero values to the front
            }
            while (insertPos < arr.length) {
                arr[insertPos++] = 0;                // fill the rest with zeroes
            }
        }
```

Complexity: Time O(n), Space O(1)

Q7. Find the Missing Number in an Array Containing 1 to n

Source: LeetCode 268 — Missing Number

Problem: Given an array holding n distinct numbers from `0..n` (one is missing), find the missing one.

Example

```
Input:  arr = [3, 0, 1], n = 3
Output: 2
Explanation: Expected sum of 0..3 is 6, actual sum is 4, so 6-4 = 2 is missing.
```

Approach: Compare the expected sum of `1..n` (or `0..n`, depending on range) against the actual sum of the array — the difference is the missing number. (XOR-ing all indices and values also works and safely avoids integer overflow on huge inputs.)

```java
static int findMissingNumber(int[] arr, int n) {
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int x : arr) actualSum += x;
    return expectedSum - actualSum;
}
```

Complexity: Time O(n), Space O(1)

Q8. Find the Duplicate Number in an Array of n+1 Integers (Values 1..n)

Source: LeetCode 287 — Find the Duplicate Number

Problem: An array of n+1 integers where every value is between 1 and n (inclusive) has exactly one repeated value. Find it **without modifying the array and using O(1) extra space**.

Example

```
Input:  arr = [1, 3, 4, 2, 2]
Output: 2
```

Approach — Floyd's Cycle Detection: Treat each value `arr[i]` as a "pointer" to the next index. Because a value repeats, following these pointers eventually loops — exactly like a linked list with a cycle (see Q67). Run the classic slow/fast pointer trick to find where the cycle begins; that meeting point is the duplicate.

> **Teaching note:** This is the same algorithm as detecting a cycle in a linked list, just applied to an array where `arr[i]` plays the role of `next`. It's a favorite because it's counter-intuitive the first time you see it, but very elegant once you connect it to Q67 and Q73.

```java
static int findDuplicate(int[] arr) {
    int slow = arr[0], fast = arr[0];
    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    } while (slow != fast);

    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }
    return slow;
}
```

Complexity: Time O(n), Space O(1)

Q9. Find All Pairs in an Array With a Given Sum (Two Sum, All Pairs)

Source: LeetCode 1 — Two Sum (LC returns one pair's indices; this GfG-style variant prints every value pair.)

Problem: Print every pair of numbers in the array that adds up to a target sum.

Example

```
Input:  arr = [1, 5, 7, -1, 5], target = 6
Output: (1, 5), (7, -1), (1, 5)
```

Approach: Scan once, keeping a `HashSet` of numbers already seen. For each new number, check whether its complement (`target - x`) is already in the set.

```java
static void pairsWithSum(int[] arr, int target) {
    Set<Integer> seen = new HashSet<>();
    for (int x : arr) {
        int need = target - x;
        if (seen.contains(need)) System.out.println(need + ", " + x);
        seen.add(x);
    }
}
```

Complexity: Time O(n), Space O(n)

Q10. Find the Intersection of Two Arrays (Common Elements, No Duplicates)

Source: LeetCode 349 — Intersection of Two Arrays

Problem: Given two arrays, return the distinct elements present in both.

Example

```
Input:  a = [1, 2, 2, 1], b = [2, 2]
Output: [2]
```

Approach: Put one array's values into a `HashSet`, then scan the second array and keep whichever values are already in that set.

```
        static List<Integer> intersection(int[] a, int[] b) {
            Set<Integer> set = new HashSet<>();
            for (int x : a) set.add(x);
            Set<Integer> result = new HashSet<>();
            for (int x : b) if (set.contains(x)) result.add(x);
            return new ArrayList<>(result);
        }
```

Complexity: Time O(n+m), Space O(n)

Q11. Find the Union of Two Arrays (Distinct Elements From Both)

Source: Not on LeetCode as its own numbered problem — classic GfG problem, natural companion to Q10.

Problem: Given two arrays, return all distinct elements present in either one.

Example

```
Input:  a = [1, 2, 3], b = [2, 3, 4]
Output: [1, 2, 3, 4]
```

Approach: Dump both arrays into one `Set`; a `LinkedHashSet` keeps first-seen order if that matters for your output.

```
        static List<Integer> union(int[] a, int[] b) {
            Set<Integer> set = new LinkedHashSet<>();
            for (int x : a) set.add(x);
            for (int x : b) set.add(x);
            return new ArrayList<>(set);
        }
```

Complexity: Time O(n+m), Space O(n+m)

Q12. Find the Majority Element (Appears More Than n/2 Times)

Source: LeetCode 169 — Majority Element

Problem: Given an array where one value occurs more than `n/2` times, find that value in O(1) extra space.

Example

```
Input:  arr = [2, 2, 1, 1, 1, 2, 2]
Output: 2
```

Approach — Boyer-Moore Voting Algorithm: Keep a `candidate` and a `count`. Walking through the array, if `count` hits 0, pick a fresh candidate; otherwise increment count when we see the candidate again, decrement when we don't.

> **Teaching note — why it works:** Think of matching votes as "cancelling out." Since the majority element appears more than half the time, it can never be fully cancelled out by all the other elements combined — so whatever survives at the end must be it. It feels like magic the first time, but it's really just: majority votes always outnumber every other vote combined.

```java
static int majorityElement(int[] arr) {
    int count = 0, candidate = 0;
    for (int x : arr) {
        if (count == 0) candidate = x;
        count += (x == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Complexity: Time O(n), Space O(1)

Q13. Find the Equilibrium Index (Left Sum Equals Right Sum)

Source: GfG classic *Equilibrium Index*; functionally identical to LeetCode 724 — Find Pivot Index.

Problem: Find an index where the sum of elements to its left equals the sum of elements to its right.

Example

```
Input:  arr = [-7, 1, 5, 2, -4, 3, 0]
Output: 3
Explanation: At index 3, left sum (-7+1+5=-1) equals right sum (-4+3+0=-1).
```

Approach: Compute the total sum first. Then walk left to right, keeping a running `leftSum`; the "right sum" at any point is just `total - leftSum - arr[i]`.

```java
static int equilibriumIndex(int[] arr) {
    int total = 0;
    for (int x : arr) total += x;

    int leftSum = 0;
    for (int i = 0; i < arr.length; i++) {
        total -= arr[i];                // total now represents the "right sum"
        if (leftSum == total) return i;
        leftSum += arr[i];
    }
    return -1;
}
```

Complexity: Time O(n), Space O(1)

Q14. Rearrange Array in Alternating Positive and Negative Order

Source: GfG classic *Rearrange array in alternating positive & negative items*; close cousin of LeetCode 2149 — Rearrange Array Elements by Sign (LC guarantees an equal split of positives/negatives; this version handles leftovers too).

Problem: Rearrange the array so that positive and negative numbers alternate, preserving their relative order within each sign group.

Example

```
Input:  arr = [1, 2, 3, -4, -1, 4]
Output: [1, -4, 2, -1, 3, 4]
```

Approach: Split into two lists (positives, negatives), then merge alternately; whichever list runs out first, append the leftover tail of the other.

```java
static void rearrangeAlternate(int[] arr) {
    List<Integer> pos = new ArrayList<>(), neg = new ArrayList<>();
    for (int x : arr) { if (x >= 0) pos.add(x); else neg.add(x); }

    int i = 0, p = 0, n = 0;
    while (p < pos.size() && n < neg.size()) {
        arr[i++] = pos.get(p++);
        arr[i++] = neg.get(n++);
    }
    while (p < pos.size()) arr[i++] = pos.get(p++);
    while (n < neg.size()) arr[i++] = neg.get(n++);
}
```

Complexity: Time O(n), Space O(n)

Q15. Find “Leaders” in an Array (Greater Than Every Element to Its Right)

Source: GfG classic *Leaders in an Array*; same underlying idea as LeetCode 1762 — Buildings With an Ocean View.

Problem: An element is a “leader” if it’s strictly greater than every element to its right. The last element is always a leader. Return all leaders, in original left-to-right order.

Example

```
Input:  arr = [16, 17, 4, 3, 5, 2]
Output: [17, 5, 2]
Explanation: 17 beats everything after it; 5 beats 2; 2 is last, so it's automatically a leader.
```

Approach: Scan from the **right**, remembering the max seen so far. Anything bigger than that running max is a new leader.

```
        static List<Integer> findLeaders(int[] arr) {
            List<Integer> leaders = new ArrayList<>();
            int maxFromRight = arr[arr.length - 1];
            leaders.add(maxFromRight);
            for (int i = arr.length - 2; i >= 0; i--) {
                if (arr[i] > maxFromRight) {
                    maxFromRight = arr[i];
                    leaders.add(maxFromRight);
                }
            }
            Collections.reverse(leaders);   // we built it right-to-left, so flip it back
            return leaders;
        }
```

Complexity: Time O(n), Space O(n)

Q16. Merge Two Sorted Arrays Into One Sorted Array

Source: LeetCode 88 — Merge Sorted Array (LC merges in place into the first array’s extra capacity; this version returns a brand-new array, which is simpler to reason about first).

Problem: Merge two already-sorted arrays into a single sorted array.

Example

```
Input:  a = [1, 3, 5], b = [2, 4, 6]
Output: [1, 2, 3, 4, 5, 6]
```

Approach: Classic two-pointer merge — the same “merge” step used inside merge sort (see Q92).

```
        static int[] mergeSorted(int[] a, int[] b) {
            int[] result = new int[a.length + b.length];
            int i = 0, j = 0, k = 0;
            while (i < a.length && j < b.length) {
                result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
            }
            while (i < a.length) result[k++] = a[i++];
            while (j < b.length) result[k++] = b[j++];
            return result;
        }
```

Complexity: Time O(n+m), Space O(n+m)

Q17. Find Min and Max in an Array Using Minimum Comparisons

Source: Not on LeetCode — classic GfG problem about comparison-count optimization.

Problem: Find both the minimum and maximum of an array using fewer than the naive `2n` comparisons.

Example

```
Input:  arr = [1000, 11, 445, 1, 330, 3000]
Output: min = 1, max = 3000
```

Approach: Process elements two at a time. Compare the pair against each other first (1 comparison), then compare only the pair's winner against the running max and only the pair's loser against the running min (2 more comparisons) — 3 comparisons per 2 elements instead of 4.

```java
        static int[] minMax(int[] arr) {
            int min, max, i;
            int n = arr.length;
            if (n % 2 == 0) {
                min = Math.min(arr[0], arr[1]);
                max = Math.max(arr[0], arr[1]);
                i = 2;
            } else {
                min = max = arr[0];
                i = 1;
            }
            while (i < n - 1) {
                int localMin = Math.min(arr[i], arr[i + 1]);
                int localMax = Math.max(arr[i], arr[i + 1]);
                min = Math.min(min, localMin);
                max = Math.max(max, localMax);
                i += 2;
            }
            return new int[]{min, max};
        }
```

Complexity: Time O(n), Space O(1)

Q18. Trapping Rain Water — Max Water Trapped Between Bars

Source: LeetCode 42 — Trapping Rain Water

Problem: Given bar heights, compute how much rainwater can be trapped between them after it rains.

Example

```
Input:  h = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

Approach: The water level above any bar `i` is limited by the *shorter* of the tallest bar to its left and the tallest bar to its right. Precompute `leftMax[]` and `rightMax[]` arrays, then water at `i` = `min(leftMax[i], rightMax[i]) - height[i]`.

> **Teaching note:** Picture standing on bar `i`. Water can only pile up as high as the *lower* of the two walls surrounding you — any higher and it just spills over that shorter wall. That's the entire idea; everything else is bookkeeping to get `leftMax` / `rightMax` efficiently. (A two-pointer version exists that does this in O(1) space, but the two-array version below is easier to reason about under time pressure.)

```java
        static int trapRainWater(int[] h) {
            int n = h.length;
            if (n == 0) return 0;

            int[] leftMax = new int[n], rightMax = new int[n];
            leftMax[0] = h[0];
            for (int i = 1; i < n; i++) leftMax[i] = Math.max(leftMax[i - 1], h[i]);

            rightMax[n - 1] = h[n - 1];
            for (int i = n - 2; i >= 0; i--) rightMax[i] = Math.max(rightMax[i + 1], h[i]);

            int water = 0;
            for (int i = 0; i < n; i++) water += Math.min(leftMax[i], rightMax[i]) - h[i];
            return water;
        }
```

Complexity: Time O(n), Space O(n)

Q19. Product of Array Except Self (Without Division)

Source: LeetCode 238 — Product of Array Except Self

Problem: Return an array where each element is the product of all other elements — without using division, in O(n) time.

Example

```
Input:  arr = [1, 2, 3, 4]
Output: [24, 12, 8, 6]
Explanation: result[0] = 2*3*4 = 24, result[1] = 1*3*4 = 12, etc.
```

Approach: For each index, the answer is `(product of everything before it) × (product of everything after it)`. Compute prefix products left-to-right into the result array, then sweep right-to-left multiplying in a running suffix product.

> **Teaching note:** The reason division is avoidable is that "everything except me" naturally splits into "everything to my left" and "everything to my right" — two totally independent passes. First pass fills `result[i]` with the prefix product (product of everything *before* i). Second pass multiplies each `result[i]` by a running suffix product built from the *right* end inward. By the time both passes are done, every cell holds prefix × suffix.

```java
static int[] productExceptSelf(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];

    result[0] = 1;
    for (int i = 1; i < n; i++) result[i] = result[i - 1] * arr[i - 1];  // prefix products

    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;     // multiply in the suffix product
        suffix *= arr[i];
    }
    return result;
}
```

Complexity: Time O(n), Space O(1) extra (excluding the output array)

Q20. Find the Peak Element (Greater Than Both Neighbours)

Source: LeetCode 162 — Find Peak Element

Problem: A peak element is strictly greater than its neighbours. Find the index of any one peak in O(log n) time.

Example

```
Input:  arr = [1, 2, 3, 1]
Output: 2
Explanation: arr[2] = 3 is greater than both arr[1]=2 and arr[3]=1.
```

Approach: Binary search: if `arr[mid] > arr[mid+1]`, a peak must exist somewhere at or before `mid` (the array is "descending" from here), so search left; otherwise search right.

```java
static int findPeakElement(int[] arr) {
    int lo = 0, hi = arr.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] > arr[mid + 1]) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Complexity: Time O(log n), Space O(1)

Q21. Sort an Array of 0s, 1s and 2s — Dutch National Flag Algorithm

Source: LeetCode 75 — Sort Colors

Problem: Sort an array containing only 0s, 1s, and 2s in a single pass, in place.

Example

```
Input:  arr = [2, 0, 2, 1, 1, 0]
Output: [0, 0, 1, 1, 2, 2]
```

Approach — three pointers: `low` marks the boundary after which all 0s are placed, `high` marks the boundary before which all 2s are placed, and `mid` is the current element being examined. Swap 0s down to `low`, leave 1s where they are, and swap 2s up to `high`.

> **Teaching note:** Think of the array as three unsorted zones that shrink as you go: `[0s | 1s | unexplored | 2s]`. `mid` explores the unknown middle zone. Seeing a 0 means it belongs in the 0-zone, so swap it to `low` and grow that zone (and since whatever came from `low` is guaranteed to be a 1, it's safe to also move `mid` forward). Seeing a 2 means it belongs at the far end, so swap it to `high` and shrink the 2-zone — but *don't* advance `mid` yet, because the element swapped in from the high end hasn't been checked. Seeing a 1 just leaves it and moves on.

```java
static void sort012(int[] arr) {
    int low = 0, mid = 0, high = arr.length - 1;
    while (mid <= high) {
        if (arr[mid] == 0) {
            int t = arr[low]; arr[low] = arr[mid]; arr[mid] = t;
            low++; mid++;
        } else if (arr[mid] == 1) {
            mid++;
        } else { // arr[mid] == 2
            int t = arr[mid]; arr[mid] = arr[high]; arr[high] = t;
            high--; // don't advance mid — recheck the swapped-in value
        }
    }
}
```

Complexity: Time O(n), Space O(1)

Q22. Find a Contiguous Subarray With a Given Sum (Non-Negative Numbers)

Source: GfG classic *Subarray with given sum* (assumes non-negative values, which is what makes the sliding window valid).

Problem: Find the start and end indices of a contiguous subarray that sums exactly to a target value.

Example

```
Input:  arr = [1, 4, 20, 3, 10, 5], target = 33
Output: [2, 4]
Explanation: arr[2..4] = 20 + 3 + 10 = 33.
```

Approach — sliding window: Expand the window on the right, adding to a running sum. Whenever the sum overshoots the target, shrink from the left until it doesn't. This only works because all numbers are non-negative — shrinking the window always decreases the sum predictably.

```java
static int[] subarrayWithSum(int[] arr, int target) {
    int start = 0, sum = 0;
    for (int end = 0; end < arr.length; end++) {
        sum += arr[end];
        while (sum > target && start < end) {
            sum -= arr[start];
            start++;
        }
        if (sum == target) return new int[]{start, end};
    }
    return new int[]{-1, -1};
}
```

Complexity: Time O(n), Space O(1)

Q23. Find the Kth Largest Element in an Array

Source: LeetCode 215 — Kth Largest Element in an Array

Problem: Find the kth largest element in an unsorted array (kth largest, not kth *distinct*).

Example

```
Input:  arr = [3, 2, 1, 5, 6, 4], k = 2
Output: 5
```

Approach: Keep a min-heap of size k. Push every element in; whenever the heap grows past size k, pop the smallest. Whatever survives at the end — the smallest thing in your "top k" pile — is the kth largest overall.

```java
static int kthLargest(int[] arr, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int x : arr) {
        minHeap.add(x);
        if (minHeap.size() > k) minHeap.poll();   // evict the smallest
    }
    return minHeap.peek();
}
```

Complexity: Time O(n log k), Space O(k)

Q24. Find Common Elements in Three Sorted Arrays

Source: GfG classic — the three-array extension of LeetCode 350 — Intersection of Two Arrays II.

Problem: Given three sorted arrays, find the elements common to all three.

Example

```
Input:  a = [1,5,10,20,40,80], b = [6,7,20,80,100], c = [3,4,15,20,30,70,80,120]
Output: [20, 80]
```

Approach: Three pointers moving together, all advancing the smallest of the three current values until a triple match is found.

```java
static List<Integer> commonInThree(int[] a, int[] b, int[] c) {
    List<Integer> result = new ArrayList<>();
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length && k < c.length) {
        if (a[i] == b[j] && b[j] == c[k]) { result.add(a[i]); i++; j++; k++; }
        else if (a[i] <= b[j] && a[i] <= c[k]) i++;
        else if (b[j] <= a[i] && b[j] <= c[k]) j++;
        else k++;
    }
    return result;
}
```

Complexity: Time O(n1+n2+n3), Space O(1) extra

Q25. Best Time to Buy and Sell Stock (Single Transaction)

Source: LeetCode 121 — Best Time to Buy and Sell Stock

Problem: Given daily stock prices, find the maximum profit from buying on one day and selling on a later day. Return 0 if no profit is possible.

Example

```
Input:  prices = [7, 1, 5, 3, 6, 4]
Output: 5
Explanation: Buy at 1 (day 2), sell at 6 (day 5), profit = 5.
```

Approach: Track the lowest price seen so far as you scan left to right; at every day, check what profit selling *today* would give if you'd bought at that lowest price.

```java
        static int maxProfit(int[] prices) {
            int minPrice = Integer.MAX_VALUE, maxProfit = 0;
            for (int p : prices) {
                minPrice = Math.min(minPrice, p);        // cheapest price seen so far
                maxProfit = Math.max(maxProfit, p - minPrice); // profit if we sold today
            }
            return maxProfit;
        }
```

Complexity: Time O(n), Space O(1)

Section 2: Strings (Q26–Q45)

Q26. Reverse a String

Source: LeetCode 344 — Reverse String

Problem: Reverse a string's characters.

Example

```
Input:  s = "hello"
Output: "olleh"
```

Approach: Convert to a char array, then two-pointer swap from both ends (same pattern as Q3).

```java
static String reverseString(String s) {
    char[] c = s.toCharArray();
    int l = 0, r = c.length - 1;
    while (l < r) {
        char t = c[l]; c[l] = c[r]; c[r] = t;
        l++; r--;
    }
    return new String(c);
}
```

Complexity: Time O(n), Space O(n) for the new String

Q27. Check if a String is a Palindrome

Source: LeetCode 125 — Valid Palindrome (LC also strips non-alphanumeric characters and ignores case; this simpler version assumes clean input).

Problem: Check whether a string reads the same forwards and backwards.

Example

```
Input:  s = "racecar"
Output: true
```

Approach: Two pointers from both ends, moving inward; bail out the moment characters don't match.

```java
static boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}
```

Complexity: Time O(n), Space O(1)

Q28. Check if Two Strings Are Anagrams

Source: LeetCode 242 — Valid Anagram

Problem: Check whether two strings are anagrams of each other (same letters, same counts, any order).

Example

```
Input:  a = "listen", b = "silent"
Output: true
```

Approach: Use a 26-length frequency array (assuming lowercase a-z). Increment for characters in the first string, decrement for the second — if they're anagrams, every count returns to 0.

```java
    static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] freq = new int[26];
        for (char c : a.toCharArray()) freq[c - 'a']++;
        for (char c : b.toCharArray()) freq[c - 'a']--;
        for (int f : freq) if (f != 0) return false;
        return true;
    }
```

Complexity: Time O(n), Space O(1) — fixed-size 26 array

Q29. Find the First Non-Repeating Character in a String

Source: LeetCode 387 — First Unique Character in a String

Problem: Return the first character in a string that doesn't repeat.

Example

```
Input:  s = "leetcode"
Output: 'l'
```

Approach: First pass builds a frequency map. Second pass walks the string *in order* and returns the first character whose count is 1 (order matters — a HashMap alone won't preserve it, which is why a second linear scan over the original string is needed).

```java
    static char firstNonRepeating(String s) {
        Map<Character, Integer> freq = new LinkedHashMap<>();
        for (char c : s.toCharArray()) freq.put(c, freq.getOrDefault(c, 0) + 1);
        for (char c : s.toCharArray()) if (freq.get(c) == 1) return c;
        return '_'; // sentinel: none found
    }
```

Complexity: Time O(n), Space O(n)

Q30. Count Occurrences of Each Character in a String

Source: Not its own LeetCode problem — a building block used inside many string problems (including Q28 and Q29 above).

Problem: Return a map of each character to how many times it appears.

Example

```
Input:  s = "aabbbc"
Output: {a=2, b=3, c=1}
```

Approach: Single pass, incrementing a HashMap count per character.

```java
    static Map<Character, Integer> countChars(String s) {
        Map<Character, Integer> freq = new LinkedHashMap<>();
        for (char c : s.toCharArray()) freq.put(c, freq.getOrDefault(c, 0) + 1);
        return freq;
    }
```

Complexity: Time O(n), Space O(n)

Q31. Remove Duplicate Characters From a String (Keep First-Occurrence Order)

Source: Not directly numbered on LeetCode at this simple level; LeetCode 316 — Remove Duplicate Letters is a much harder cousin that also enforces lexicographically smallest ordering.

Problem: Remove duplicate characters, keeping only the first occurrence of each, in original order.

Example

```
Input:  s = "programming"
Output: "progamin"
```

Approach: A `LinkedHashSet` naturally remembers insertion order and silently ignores duplicates — perfect fit here.

```
        static String removeDuplicates(String s) {
            StringBuilder sb = new StringBuilder();
            Set<Character> seen = new LinkedHashSet<>();
            for (char c : s.toCharArray()) {
                if (seen.add(c)) sb.append(c);   // add() returns false if already present
            }
            return sb.toString();
        }
```

Complexity: Time O(n), Space O(n)

Q32. Check if a String Contains Only Digits (Input Validation)

Source: Not its own LeetCode problem — a common validation helper (a simplified piece of what LeetCode 8 — String to Integer (atoi) needs internally).

Problem: Return true only if every character in the string is a digit (and the string isn't empty).

Example

```
Input:  s = "12345"
Output: true
```

Approach: Scan every character and confirm it's a digit.

```
        static boolean isNumeric(String s) {
            if (s.isEmpty()) return false;
            for (char c : s.toCharArray()) {
                if (!Character.isDigit(c)) return false;
            }
            return true;
        }
```

Complexity: Time O(n), Space O(1)

Q33. Find All Permutations of a String

Source: LeetCode 46 — Permutations (LC works on an int array; the same backtracking pattern applies directly to a char array).

Problem: Generate every possible arrangement (permutation) of a string's characters.

Example

```
Input:  s = "abc"
Output: ["abc", "acb", "bac", "bca", "cab", "cba"]
```

Approach — backtracking: At each position, try placing every remaining character there (by swapping it in), recurse to fill the rest of the string, then swap back ("backtrack") to restore the array before trying the next candidate.

> **Teaching note:** The swap-recurse-swap-back pattern is the heart of most backtracking problems. `index` marks "everything before this position is fixed." For each position, we try every unused character in that slot by swapping it into place, solve the rest recursively, then undo the swap so the next candidate for this slot starts from a clean array.

```
        static void permute(char[] arr, int index, List<String> result) {
            if (index == arr.length) {
                result.add(new String(arr));
                return;
            }
            for (int i = index; i < arr.length; i++) {
                char t = arr[index]; arr[index] = arr[i]; arr[i] = t;     // choose
                permute(arr, index + 1, result);                          // explore
                t = arr[index]; arr[index] = arr[i]; arr[i] = t;          // un-choose (backtrack)
            }
        }
```

Complexity: Time O(n × n!), Space O(n) recursion stack (excluding output storage)

Q34. Longest Substring Without Repeating Characters

Source: LeetCode 3 — Longest Substring Without Repeating Characters

Problem: Find the length of the longest substring that has no repeated characters.

Example

```
Input:  s = "abcabcbb"
Output: 3
Explanation: "abc" is the longest substring without repeats.
```

Approach — sliding window with last-seen index: Keep a window `[start, end]` and a map of each character's most recent index. When we hit a character already in the window, jump `start` to just past its last occurrence.

> **Teaching note:** The key subtlety is the check `lastSeen.get(c) >= start`. Without it, you'd wrongly shrink the window because of a character that repeated *before* the current window even started (i.e., it's stale information, already outside our valid range). Only jump `start` forward if the repeat is actually *inside* the current window.

```java
static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int maxLen = 0, start = 0;
    for (int end = 0; end < s.length(); end++) {
        char c = s.charAt(end);
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= start) {
            start = lastSeen.get(c) + 1;   // jump past the earlier occurrence
        }
        lastSeen.put(c, end);
        maxLen = Math.max(maxLen, end - start + 1);
    }
    return maxLen;
}
```

Complexity: Time O(n), Space O(min(n, charset size))

Q35. Find the Longest Common Prefix Among an Array of Strings

Source: LeetCode 14 — Longest Common Prefix

Problem: Find the longest string that is a prefix of every string in the array.

Example

```
Input:  strs = ["flower", "flow", "flight"]
Output: "fl"
```

Approach: Start by assuming the first string is the answer, then keep trimming its end until every other string actually starts with it.

```java
static String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (!strs[i].startsWith(prefix)) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}
```

Complexity: Time O(n × m) where m is the shortest string length, Space O(1)

Q36. Check if One String is a Rotation of Another

Source: LeetCode 796 — Rotate String

Problem: Check whether string `b` is a rotation of string `a` (e.g., “erbottlewat” is a rotation of “waterbottle”).

Example

```
Input:  a = "waterbottle", b = "erbottlewat"
Output: true
```

Approach: The neat trick — if `b` really is a rotation of `a`, then `b` must appear somewhere inside `a + a` (concatenating `a` with itself covers every possible rotation point).

```java
static boolean isRotation(String a, String b) {
    if (a.length() != b.length()) return false;
    return (a + a).contains(b);
}
```

Complexity: Time O(n) (Java’s `contains` uses an efficient substring search), Space O(n)

Q37. Reverse the Words in a Sentence

Source: LeetCode 151 — Reverse Words in a String

Problem: Reverse the order of words in a sentence, while keeping each word’s own characters intact.

Example

```
Input:  s = "the sky is blue"
Output: "blue is sky the"
```

Approach: Split on whitespace, then rebuild the sentence appending words from last to first.

```java
static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i != 0) sb.append(" ");
    }
    return sb.toString();
}
```

Complexity: Time O(n), Space O(n)

Q38. Find the Longest Palindromic Substring

Source: LeetCode 5 — Longest Palindromic Substring

Problem: Find the longest substring that is itself a palindrome.

Example

```
Input:  s = "babad"
Output: "bab"  (or "aba" — both are valid answers)
```

Approach — expand around center: Every palindrome has a center. Try every possible center (each single character for odd-length palindromes, and every gap between two characters for even-length ones), and expand outward from it as long as both sides match.

> **Teaching note:** There are `2n - 1` possible centers in a string of length n (n single-character centers + n-1 between-character centers). Expanding from each one costs at most O(n), giving O(n²) total — much simpler to code correctly under exam pressure than the O(n) Manacher’s algorithm, and plenty fast enough for typical Accenture-style constraints.

```java
static String longestPalindrome(String s) {
    if (s.isEmpty()) return "";
    int start = 0, maxLen = 1;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);       // odd-length palindrome centered at i
        int len2 = expand(s, i, i + 1);   // even-length palindrome centered between i, i+1
        int len = Math.max(len1, len2);
        if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
        }
    }
    return s.substring(start, start + maxLen);
}
static int expand(String s, int l, int r) {
    while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
    return r - l - 1;   // length of the palindrome found
}
```

Complexity: Time O(n²), Space O(1)

Q39. Convert a String to an Integer (Implement atoi)

Source: LeetCode 8 — String to Integer (atoi)

Problem: Parse a string into a 32-bit signed integer, handling leading whitespace, an optional sign, and stopping at the first non-digit character. Clamp to `Integer.MAX_VALUE` / `MIN_VALUE` on overflow.

Example

```
Input:  s = "   -42"
Output: -42
```

Approach: Trim whitespace, capture an optional sign, then accumulate digits one at a time — checking for overflow at every step, not just at the end.

```java
static int myAtoi(String s) {
    s = s.trim();
    if (s.isEmpty()) return 0;
    int i = 0, sign = 1;
    long result = 0;
    if (s.charAt(0) == '+' || s.charAt(0) == '-') {
        sign = (s.charAt(0) == '-') ? -1 : 1;
        i++;
    }
    while (i < s.length() && Character.isDigit(s.charAt(i))) {
        result = result * 10 + (s.charAt(i) - '0');
        if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE;
        if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
        i++;
    }
    return (int) (result * sign);
}
```

Complexity: Time O(n), Space O(1)

Q40. Check if a String of Parentheses/Brackets is Balanced

Source: LeetCode 20 — Valid Parentheses

Problem: Given a string containing only `()`, `{}`, `[]`, determine if the brackets are properly matched and nested.

Example

```
Input:  s = "{[()]}"
Output: true
Input:  s = "{[(])}"
Output: false
```

Approach — stack: Push every opening bracket. On a closing bracket, pop the stack and check it matches the expected opening bracket. If the stack is empty when we need to pop, or anything is left on the stack at the end, it’s unbalanced.

```java
    static boolean isValidParentheses(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')' && top != '(') ||
                    (c == '}' && top != '{') ||
                    (c == ']' && top != '[')) return false;
            }
        }
        return stack.isEmpty();
    }
```

Complexity: Time O(n), Space O(n)

Q41. Count Vowels and Consonants in a String

Source: Not on LeetCode — a common warm-up/validation-style GfG question.

Problem: Count how many vowels and how many consonants appear in a string (ignoring non-letter characters).

Example

```
Input:  s = "Hello World"
Output: vowels = 3, consonants = 7
```

Approach: Single pass; check letter membership in a small vowel set.

```java
    static int[] countVowelsConsonants(String s) {
        int vowels = 0, consonants = 0;
        String vSet = "aeiouAEIOU";
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
                if (vSet.indexOf(c) != -1) vowels++;
                else consonants++;
            }
        }
        return new int[]{vowels, consonants};
    }
```

Complexity: Time O(n), Space O(1)

Q42. Find the Length of the Longest Word in a Sentence

Source: Not on LeetCode — classic GfG warm-up.

Problem: Given a sentence, find the length of its longest word.

Example

```
Input:  sentence = "The quick brown fox jumped"
Output: 6
Explanation: "quick" and "jumped" both have 6 letters.
```

Approach: Split on whitespace, track the max length seen.

```java
    static int longestWordLength(String sentence) {
        String[] words = sentence.trim().split("\\s+");
        int maxLen = 0;
        for (String w : words) maxLen = Math.max(maxLen, w.length());
        return maxLen;
    }
```

Complexity: Time O(n), Space O(n)

Q43. Compress a String Using Run-Length Encoding

Source: Similar in spirit to LeetCode 443 — String Compression, though LC's exact spec compresses in place inside a char array — this version returns a new string.

Problem: Replace consecutive repeated characters with the character followed by its count (e.g., "aaabb" → "a3b2"). Return the original string if compression doesn't actually shrink it.

Example

```
Input:  s = "aaabbbccd"
Output: "a3b3c2d1"
```

Approach: Walk through the string counting consecutive runs of the same character, appending `char + count` for each run.

```
        static String compressString(String s) {
            StringBuilder sb = new StringBuilder();
            int n = s.length(), i = 0;
            while (i < n) {
                char c = s.charAt(i);
                int count = 0;
                while (i < n && s.charAt(i) == c) { count++; i++; }
                sb.append(c).append(count);
            }
            return sb.length() < s.length() ? sb.toString() : s;
        }
```

Complexity: Time O(n), Space O(n)

Q44. Find the Smallest Window in a String Containing All Characters of Another String

Source: LeetCode 76 — Minimum Window Substring

Problem: Given strings `s` and `t`, find the smallest substring of `s` that contains every character of `t` (including duplicates).

Example

```
Input:  s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
```

Approach — sliding window with two frequency maps: Expand the right edge until the window contains everything `t` needs (`formed == required`). Then greedily shrink from the left, recording the smallest valid window found, until the window becomes invalid again — then resume expanding.

> **Teaching note:** This is one of the trickiest sliding-window problems because the window doesn't just grow — it *alternates* between growing (to become valid) and shrinking (to become minimal while staying valid). `formed` tracks how many distinct required characters currently have *enough* copies in the window; only when `formed == required` do we even attempt to shrink. Practicing this pattern makes every other sliding-window problem feel easy by comparison.

```java
static String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.put(c, need.getOrDefault(c, 0) + 1);

    Map<Character, Integer> window = new HashMap<>();
    int required = need.size(), formed = 0;
    int l = 0, minLen = Integer.MAX_VALUE, minStart = 0;

    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        window.put(c, window.getOrDefault(c, 0) + 1);
        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue()) formed++;

        while (formed == required) {
            if (r - l + 1 < minLen) { minLen = r - l + 1; minStart = l; }
            char lc = s.charAt(l);
            window.put(lc, window.get(lc) - 1);
            if (need.containsKey(lc) && window.get(lc) < need.get(lc)) formed--;
            l++;
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLen);
}
```

Complexity: Time O(n+m), Space O(n+m)

Q45. Check if Two Strings Are at Most One Edit (Insert/Delete/Replace) Apart

Source: LeetCode 161 — *One Edit Distance* (this problem is on LeetCode's premium/locked list for some accounts — GfG has the same problem freely available as "One edit distance").

Problem: Determine whether two strings can be made identical with at most one single-character insert, delete, or replace.

Example

```
Input:  a = "pale", b = "ple"
Output: true   (delete 'a' from "pale")
```

Approach: If the length difference is more than 1, it's immediately false. Otherwise walk both strings together; the first mismatch is allowed once — if the strings are equal length treat it as a replace (advance both), if lengths differ treat it as an insert/delete (advance only the longer one). A second mismatch means false.

```java
static boolean isOneEditDistance(String a, String b) {
    int la = a.length(), lb = b.length();
    if (Math.abs(la - lb) > 1) return false;

    String shorter = la <= lb ? a : b, longer = la <= lb ? b : a;
    int i = 0, j = 0;
    boolean edited = false;
    while (i < shorter.length() && j < longer.length()) {
        if (shorter.charAt(i) != longer.charAt(j)) {
            if (edited) return false;          // already used our one allowed edit
            edited = true;
            if (shorter.length() == longer.length()) i++;   // treat as a "replace"
        } else {
            i++;
        }
        j++;
    }
    return true;
}
```

Complexity: Time O(n), Space O(1)

Section 3: Binary Search (Q46–Q57)

Q46. Binary Search on a Sorted Array (Iterative)

Source: LeetCode 704 — Binary Search

Problem: Given a sorted array, find the index of a target value (or -1 if absent) in O(log n) time.

Example

```
Input:  arr = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
```

Approach: Repeatedly compare the middle element to the target and eliminate the half that can't contain it.

> **Teaching note:** The reason binary search works at all is that the array is sorted — this single property lets us discard *half* the remaining search space with every comparison, which is why it's O(log n) instead of O(n). Always use `lo + (hi - lo) / 2` instead of `(lo + hi) / 2` — the latter can silently overflow on very large arrays.

```java
static int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;   // avoids overflow vs (lo+hi)/2
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Complexity: Time O(log n), Space O(1)

Q47. Binary Search on a Sorted Array (Recursive)

Source: LeetCode 704 — Binary Search (same problem, recursive implementation)

Problem: Same as Q46, expressed recursively — good to know both forms, since interviewers sometimes specifically ask for one or the other.

Example

```
Input:  arr = [-1, 0, 3, 5, 9, 12], target = 2
Output: -1
```

```java
static int binarySearchRec(int[] arr, int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearchRec(arr, mid + 1, hi, target);
    return binarySearchRec(arr, lo, mid - 1, target);
}
```

Complexity: Time O(log n), Space O(log n) recursion stack

Q48. Find the First and Last Occurrence of an Element in a Sorted Array

Source: LeetCode 34 — Find First and Last Position of Element in Sorted Array

Problem: Given a sorted array with duplicates, find the first and last index of a target value.

Example

```
Input:  arr = [5, 7, 7, 8, 8, 10], target = 8
Output: [3, 4]
```

Approach: Run two slightly-biased binary searches. To find the *first* occurrence, when you find a match keep searching left (`hi = mid - 1`) to see if there's an earlier one. To find the *last*, keep searching right (`lo = mid + 1`) instead.

```java
static int[] firstAndLast(int[] arr, int target) {
    return new int[]{findBound(arr, target, true), findBound(arr, target, false)};
}
static int findBound(int[] arr, int target, boolean findFirst) {
    int lo = 0, hi = arr.length - 1, result = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) {
            result = mid;
            if (findFirst) hi = mid - 1;    // keep looking left for an earlier match
            else lo = mid + 1;              // keep looking right for a later match
        } else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return result;
}
```

Complexity: Time O(log n), Space O(1)

Q49. Find the Floor and Ceil of a Number in a Sorted Array

Source: GfG classic *Floor and Ceil in a sorted array*; closely related to LeetCode 35 — Search Insert Position.

Problem: Floor = the largest element ≤ target; Ceil = the smallest element ≥ target.

Example

```
Input:  arr = [1, 2, 8, 10, 10, 12, 19], target = 5
Output: floor = 2, ceil = 8
```

Approach: Standard binary search, but instead of giving up on a miss, remember the closest candidate on each side as you narrow down.

```java
static int[] floorCeil(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1, floor = -1, ceil = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return new int[]{arr[mid], arr[mid]};
        else if (arr[mid] < target) { floor = arr[mid]; lo = mid + 1; }
        else { ceil = arr[mid]; hi = mid - 1; }
    }
    return new int[]{floor, ceil};
}
```

Complexity: Time O(log n), Space O(1)

Q50. Search in a Rotated Sorted Array (No Duplicates)

Source: LeetCode 33 — Search in Rotated Sorted Array

Problem: A sorted array has been rotated at some unknown pivot (e.g., `[4,5,6,7,0,1,2]`). Find a target's index in O(log n).

Example

```
Input:  arr = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Approach: Even after rotation, **at least one half** of `[lo, mid]` or `[mid, hi]` is always properly sorted. Figure out which half is sorted by comparing `arr[lo]` to `arr[mid]`, then check whether the target falls inside that sorted half's range — if yes, search there; if no, it must be in the other (unsorted-looking) half.

> **Teaching note:** This is the classic "rotated array" pattern that trips people up. The trick isn't finding the pivot first — it's realizing you don't need to. Just ask "is the left half sorted?" If `arr[lo] <= arr[mid]`, yes; check if target lies in `[arr[lo], arr[mid])`. Otherwise the *right* half must be sorted instead; check if target lies in `(arr[mid], arr[hi]]`. Either way, you eliminate half the array each time.

```java
static int searchRotated(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;

        if (arr[lo] <= arr[mid]) {   // left half [lo..mid] is sorted
            if (arr[lo] <= target && target < arr[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                     // right half [mid..hi] is sorted
            if (arr[mid] < target && target <= arr[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Complexity: Time O(log n), Space O(1)

Q51. Find the Integer Square Root of a Number Using Binary Search

Source: LeetCode 69 — Sqrt(x)

Problem: Compute `floor(sqrt(n))` without using `Math.sqrt`.

Example

```
Input:  n = 8
Output: 2
Explanation: sqrt(8) ≈ 2.83, floor is 2.
```

Approach — binary search on the *answer*, not the array: Search the range `[1, n]` for the largest `x` such that `x*x <= n`. This "binary search the answer space" pattern applies to many optimization problems, not just array searches.

```java
static int sqrtBinarySearch(int n) {
    if (n < 2) return n;
    long lo = 1, hi = n, ans = 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        if (mid * mid <= n) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return (int) ans;
}
```

Complexity: Time O(log n), Space O(1)

Q52. Find a Peak Element Using Binary Search

Source: LeetCode 162 — Find Peak Element — same problem as Q20, included again since interviewers group it under "Binary Search" specifically.

Solution is identical to Q20's `findPeakElement`. It's worth being able to explain *why* binary search applies here even though the array isn't fully sorted: comparing `arr[mid]` to `arr[mid+1]` always tells you which direction has a guaranteed peak, which is all binary search needs.

Q53. Count Occurrences of a Number in a Sorted Array

Source: GfG classic; directly builds on LeetCode 34 (Q48).

Problem: Count how many times a target value appears in a sorted array.

Example

```
Input:  arr = [1, 2, 2, 2, 3], target = 2
Output: 3
```

Approach: Reuse Q48's first/last occurrence search — the count is simply `last - first + 1`.

```java
static int countOccurrences(int[] arr, int target) {
    int[] range = firstAndLast(arr, target);
    if (range[0] == -1) return 0;
    return range[1] - range[0] + 1;
}
```

Complexity: Time O(log n), Space O(1)

Q54. Find the Smallest Missing Positive Number in a Sorted Array

Source: GfG classic (assumes a sorted array of distinct positives — this is a much more specific and easier setup than LeetCode 41 — First Missing Positive, which works on unsorted arrays and needs a different O(n) technique).

Problem: Given a sorted array of distinct positive integers, find the smallest positive integer missing from it.

Example

```
Input:  arr = [1, 2, 3, 5]
Output: 4
```

Approach: In a "perfect" array with no gaps, `arr[i]` would equal `i + 1`. Binary search for the first index where that pattern breaks — everything before it matches perfectly, so the answer is `lo + 1`.

```java
static int smallestMissing(int[] arr) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == mid + 1) lo = mid + 1;    // still matches, no gap yet
        else hi = mid - 1;
    }
    return lo + 1;
}
```

Complexity: Time O(log n), Space O(1)

Q55. Find the Element That Appears Once (All Others Appear Twice) in a Sorted Array

Source: LeetCode 540 — Single Element in a Sorted Array

Problem: In a sorted array where every element appears exactly twice except one, find the single one — in O(log n).

Example

```
Input:  arr = [1, 1, 2, 3, 3, 4, 4, 8, 8]
Output: 2
```

Approach: Before the single element, pairs line up at (even, odd) index positions: `arr[0]==arr[1]`, `arr[2]==arr[3]`, etc. After the single element throws things off, pairs shift to (odd, even). Binary search for where that pattern flips.

> **Teaching note:** Force `mid` to always be even first. If `arr[mid] == arr[mid+1]`, the pairing pattern is still intact up through `mid+1`, so the single element must be further right — jump past both (`lo = mid + 2`). If they *don't* match, the pattern already broke at or before `mid`, so the single element is at `mid` or to its left (`hi = mid`).

```java
static int singleNonDuplicate(int[] arr) {
    int lo = 0, hi = arr.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (mid % 2 == 1) mid--;    // ensure mid is always even
        if (arr[mid] == arr[mid + 1]) lo = mid + 2;
        else hi = mid;
    }
    return arr[lo];
}
```

Complexity: Time O(log n), Space O(1)

Q56. Search a Target in a 2D Matrix (Sorted Overall, Row-Wise Then Column-Wise)

Source: LeetCode 74 — Search a 2D Matrix

Problem: Each row is sorted, and the first element of each row is greater than the last element of the previous row (so the whole matrix reads like one giant sorted array). Find whether a target exists.

Example

```
Input:  matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Approach: Since the matrix behaves like a flattened 1D sorted array of size `rows × cols`, binary search over indices `0..rows*cols-1`, converting each mid index back into `(row, col)` with simple division/modulo.

```java
static boolean searchMatrix(int[][] matrix, int target) {
    if (matrix.length == 0) return false;
    int rows = matrix.length, cols = matrix[0].length;
    int lo = 0, hi = rows * cols - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int val = matrix[mid / cols][mid % cols];   // convert flat index back to (row, col)
        if (val == target) return true;
        else if (val < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return false;
}
```

Complexity: Time O(log(rows × cols)), Space O(1)

Q57. Find the Minimum Element in a Rotated Sorted Array

Source: LeetCode 153 — Find Minimum in Rotated Sorted Array

Problem: A sorted array was rotated at an unknown point. Find the minimum element in O(log n).

Example

```
Input:  arr = [4, 5, 6, 7, 0, 1, 2]
Output: 0
```

Approach: Compare `arr[mid]` to `arr[hi]`. If `arr[mid] > arr[hi]`, the rotation point (and thus the minimum) must be to the right of `mid`. Otherwise it's at `mid` or to its left.

```java
static int findMinRotated(int[] arr) {
    int lo = 0, hi = arr.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] > arr[hi]) lo = mid + 1;
        else hi = mid;
    }
    return arr[lo];
}
```

Complexity: Time O(log n), Space O(1)

Section 4: Matrices / 2D Arrays (Q58-Q65)

Q58. Find the Transpose of a Matrix

Source: LeetCode 867 — Transpose Matrix

Problem: Flip a matrix over its main diagonal, turning rows into columns.

Example

```
Input:  [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]
```

Approach: Build a new `cols × rows` matrix, copying `matrix[i][j]` to `result[j][i]`.

```
        static int[][] transpose(int[][] matrix) {
            int rows = matrix.length, cols = matrix[0].length;
            int[][] result = new int[cols][rows];
            for (int i = 0; i < rows; i++)
                for (int j = 0; j < cols; j++)
                    result[j][i] = matrix[i][j];
            return result;
        }
```

Complexity: Time O(rows × cols), Space O(rows × cols)

Q59. Rotate a Square Matrix by 90 Degrees Clockwise, In Place

Source: LeetCode 48 — Rotate Image

Problem: Rotate an n×n matrix 90° clockwise, without allocating another matrix.

Example

```
Input:  [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```

Approach — transpose, then reverse each row: Transposing flips the matrix over its main diagonal; reversing each row then completes the 90° clockwise rotation.

> **Teaching note:** It helps to verify this on paper once: transpose `[[1,2,3],[4,5,6],[7,8,9]]` → `[[1,4,7],[2,5,8],[3,6,9]]`, then reverse each row → `[[7,4,1],[8,5,2],[9,6,3]]` — exactly the 90° clockwise rotation. Two simple, well-known operations combine into a trickier one; this “compose simple operations” habit is worth having for matrix problems generally.

```
        static void rotate90(int[][] matrix) {
            int n = matrix.length;
            // step 1: transpose (swap across the main diagonal)
            for (int i = 0; i < n; i++)
                for (int j = i + 1; j < n; j++) {
                    int t = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix[j][i] = t;
                }
            // step 2: reverse each row
            for (int[] row : matrix) {
                int l = 0, r = row.length - 1;
                while (l < r) { int t = row[l]; row[l] = row[r]; row[r] = t; l++; r--; }
            }
        }
```

Complexity: Time O(n²), Space O(1)

Q60. Print a Matrix in Spiral Order

Source: LeetCode 54 — Spiral Matrix

Problem: Traverse a matrix in a clockwise spiral, starting from the top-left, and return the visited values in order.

Example

```
Input:  [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Approach: Maintain four shrinking boundaries — `top`, `bottom`, `left`, `right`. Traverse the top row left→right, the right column top→bottom, the bottom row right→left, and the left column bottom→top, tightening each boundary after its pass, and repeat until the boundaries cross.

> **Teaching note:** The two `if` guards (`if (top <= bottom)`, `if (left <= right)`) before the last two passes are the part people forget — they prevent re-visiting a row or column that the earlier passes already fully consumed once the spiral has shrunk down to a single row or column.

```java
static List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();
    if (matrix.length == 0) return result;
    int top = 0, bottom = matrix.length - 1, left = 0, right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) result.add(matrix[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) result.add(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) result.add(matrix[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) result.add(matrix[i][left]);
            left++;
        }
    }
    return result;
}
```

Complexity: Time O(rows × cols), Space O(1) extra (excluding output)

Q61. Search an Element in a Matrix Sorted Row-Wise and Column-Wise

Source: LeetCode 240 — Search a 2D Matrix II

Problem: Each row is sorted left-to-right and each column is sorted top-to-bottom (unlike Q56, rows aren't guaranteed to chain together). Find whether a target exists.

Example

```
Input:  matrix = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target = 5
Output: true
```

Approach — start at the top-right corner: From there, every move is informative: if the current value is too big, the entire column below is also too big, so move left; if too small, the entire row to the left is also too small, so move down.

> **Teaching note:** The top-right corner is special because it's the *maximum* of its row and the *minimum* of its column simultaneously — the only cell in the matrix with that property. That's exactly what makes a clean binary elimination possible: every comparison discards either a whole row or a whole column.

```java
static boolean searchSortedMatrix(int[][] matrix, int target) {
    if (matrix.length == 0) return false;
    int row = 0, col = matrix[0].length - 1;   // start top-right
    while (row < matrix.length && col >= 0) {
        if (matrix[row][col] == target) return true;
        else if (matrix[row][col] > target) col--;   // too big, eliminate this column
        else row++;                                  // too small, eliminate this row
    }
    return false;
}
```

Complexity: Time O(rows + cols), Space O(1)

Q62. Find the Sum of Both Diagonals of a Square Matrix

Source: LeetCode 1572 — Matrix Diagonal Sum

Problem: Sum the main diagonal and the anti-diagonal of a square matrix (don't double-count the center cell in an odd-sized matrix).

Example

```
Input:  [[1,2,3],[4,5,6],[7,8,9]]
Output: 25
Explanation: main diagonal 1+5+9=15, anti-diagonal 3+5+7=15, center 5 counted once → 25.
```

```java
static int diagonalSum(int[][] matrix) {
    int n = matrix.length, sum = 0;
    for (int i = 0; i < n; i++) {
        sum += matrix[i][i];                    // main diagonal
        if (i != n - 1 - i) sum += matrix[i][n - 1 - i];  // anti-diagonal, skip center
    }
    return sum;
}
```

Complexity: Time O(n), Space O(1)

Q63. Set Matrix Zeroes — If an Element is 0, Zero Out Its Entire Row and Column

Source: LeetCode 73 — Set Matrix Zeroes

Problem: Given an m×n matrix, if any cell is 0, set its entire row and column to 0 — in place, ideally with O(1) extra space.

Example

```
Input:  [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]
```

Approach: Instead of a separate boolean grid to remember which rows/columns need zeroing, reuse the matrix's own first row and first column as markers. Since that overwrites information about whether the first row/column *themselves* need zeroing, save those two facts in separate flags first.

> **Teaching note:** This is a genuinely tricky O(1)-space trick. Walk through it in three passes: (1) check up front whether the first row or first column originally contains a zero, storing that in two booleans; (2) scan the rest of the matrix, and whenever `matrix[i][j]==0`, mark `matrix[i][0]=0` and `matrix[0][j]=0` as flags; (3) scan again (skipping row/col 0) and zero out any cell whose row-flag or column-flag is set; finally, zero the first row/column too if the booleans from step 1 say to.

```java
static void setZeroes(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length;
    boolean firstRowZero = false, firstColZero = false;

    for (int j = 0; j < cols; j++) if (matrix[0][j] == 0) firstRowZero = true;
    for (int i = 0; i < rows; i++) if (matrix[i][0] == 0) firstColZero = true;

    // use row 0 / col 0 as marker flags
    for (int i = 1; i < rows; i++)
        for (int j = 1; j < cols; j++)
            if (matrix[i][j] == 0) { matrix[i][0] = 0; matrix[0][j] = 0; }

    // zero out cells based on the markers
    for (int i = 1; i < rows; i++)
        for (int j = 1; j < cols; j++)
            if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;

    if (firstRowZero) for (int j = 0; j < cols; j++) matrix[0][j] = 0;
    if (firstColZero) for (int i = 0; i < rows; i++) matrix[i][0] = 0;
}
```

Complexity: Time O(rows × cols), Space O(1)

Q64. Multiply Two Matrices

Source: Standard algorithm; LeetCode 311 — Sparse Matrix Multiplication is the closest numbered LeetCode variant (optimized for matrices with many zeroes).

Problem: Multiply an `a × b` matrix by a `b × c` matrix to produce an `a × c` result.

Example

```
Input:  A = [[1,2],[3,4]], B = [[5,6],[7,8]]
Output: [[19,22],[43,50]]
```

Approach: Standard triple-nested-loop multiplication — `result[i][j]` is the dot product of row `i` of A and column `j` of B.

```java
static int[][] multiplyMatrices(int[][] a, int[][] b) {
    int rowsA = a.length, colsA = a[0].length, colsB = b[0].length;
    int[][] result = new int[rowsA][colsB];
    for (int i = 0; i < rowsA; i++)
        for (int j = 0; j < colsB; j++)
            for (int k = 0; k < colsA; k++)
                result[i][j] += a[i][k] * b[k][j];
    return result;
}
```

Complexity: Time O(n³) for n×n matrices, Space O(n²)

Q65. Check if a Matrix is an Identity Matrix

Source: Not on LeetCode — basic warm-up matrix check.

Problem: Return true if every diagonal element is 1 and every off-diagonal element is 0.

Example

```
Input:  [[1,0,0],[0,1,0],[0,0,1]]
Output: true
```

```java
static boolean isIdentityMatrix(int[][] matrix) {
    int n = matrix.length;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            if (i == j && matrix[i][j] != 1) return false;
            if (i != j && matrix[i][j] != 0) return false;
        }
    return true;
}
```

Complexity: Time O(n²), Space O(1)

Section 5: Linked List (Q66–Q75)

All questions below assume this standard singly linked list node:

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

Q66. Reverse a Linked List (Iterative)

Source: LeetCode 206 — Reverse Linked List

Problem: Reverse a singly linked list in place.

Example

```
Input:  1 -> 2 -> 3 -> 4 -> 5 -> null
Output: 5 -> 4 -> 3 -> 2 -> 1 -> null
```

Approach: Walk the list once, tracking `prev`, `curr`, and `next`. At each node, flip its `next` pointer to point backward, then shuffle all three pointers forward.

> **Teaching note:** Before touching `curr.next`, always save it into `next` first — otherwise you lose your only way to continue walking forward once you've overwritten the pointer. This save-before-you-break-it habit is essential for almost every linked-list manipulation problem.

```
static ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;   // save before we overwrite curr.next
        curr.next = prev;            // reverse the pointer
        prev = curr;
        curr = next;
    }
    return prev;   // prev is the new head
}
```

Complexity: Time O(n), Space O(1)

Q67. Detect a Cycle in a Linked List — Floyd's Cycle Detection

Source: LeetCode 141 — Linked List Cycle

Problem: Determine whether a linked list has a cycle.

Example

```
Input:  1 -> 2 -> 3 -> 4 -> back to 2 (cycle)
Output: true
```

Approach — slow/fast pointers ("tortoise and hare"): Move `slow` one step at a time and `fast` two steps at a time. If there's a cycle, `fast` will eventually lap `slow` and they'll meet; if there's no cycle, `fast` simply reaches `null` first.

> **Teaching note:** Think of a circular running track: a faster runner will always eventually catch up to and pass a slower one if they keep looping — they *must* meet at some point. If the track isn't circular (no cycle), the faster runner just reaches the finish line (`null`) and stops. Q8 (Find the Duplicate Number) and Q73 (Detect and Remove a Loop) both reuse this exact idea.

```java
static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Complexity: Time O(n), Space O(1)

Q68. Find the Middle of a Linked List

Source: LeetCode 876 — Middle of the Linked List

Problem: Find the middle node of a linked list in one pass (for even length, return the second of the two middle nodes).

Example

```
Input:  1 -> 2 -> 3 -> 4 -> 5
Output: 3
```

Approach: Same slow/fast pointer idea as Q67 — when `fast` reaches the end, `slow` has covered exactly half the distance.

```java
static ListNode findMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Complexity: Time O(n), Space O(1)

Q69. Merge Two Sorted Linked Lists

Source: LeetCode 21 — Merge Two Sorted Lists

Problem: Merge two sorted linked lists into one sorted list by splicing existing nodes together.

Example

```
Input:  l1 = 1->2->4, l2 = 1->3->4
Output: 1->1->2->3->4->4
```

Approach: Use a dummy head node so you never have to special-case “what’s the first node” — just always attach the smaller of the two current nodes and move the tail pointer forward.

```java
static ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);   // placeholder — dummy.next will be the real head
    ListNode tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
        else { tail.next = l2; l2 = l2.next; }
        tail = tail.next;
    }
    tail.next = (l1 != null) ? l1 : l2;   // attach whatever's left
    return dummy.next;
}
```

Complexity: Time O(n+m), Space O(1)

Q70. Remove Duplicates From a Sorted Linked List

Source: LeetCode 83 — Remove Duplicates from Sorted List

Problem: Given a sorted linked list, remove duplicate values so each value appears only once.

Example

```
Input:  1 -> 1 -> 2 -> 3 -> 3
Output: 1 -> 2 -> 3
```

Approach: Because the list is sorted, duplicates are always adjacent — just compare each node to the next and skip over matches.

```
        static ListNode deleteDuplicates(ListNode head) {
            ListNode curr = head;
            while (curr != null && curr.next != null) {
                if (curr.val == curr.next.val) curr.next = curr.next.next;   // skip the duplicate
                else curr = curr.next;
            }
            return head;
        }
```

Complexity: Time O(n), Space O(1)

Q71. Find the Nth Node From the End of a Linked List

Source: GfG classic; the removal version of this idea is LeetCode 19 — Remove Nth Node From End of List.

Problem: Return the value n positions from the end, in a single pass (without first counting the list's length).

Example

```
Input:  1 -> 2 -> 3 -> 4 -> 5, n = 2
Output: 4
```

Approach — two pointers, offset by n: Advance a `first` pointer n steps ahead of `second`. Then move both together; when `first` reaches the end, `second` is exactly n nodes from the end.

```
        static ListNode nthFromEnd(ListNode head, int n) {
            ListNode first = head, second = head;
            for (int i = 0; i < n; i++) {
                if (first == null) return null;   // list shorter than n
                first = first.next;
            }
            while (first != null) {
                first = first.next;
                second = second.next;
            }
            return second;
        }
```

Complexity: Time O(n), Space O(1)

Q72. Check if a Linked List is a Palindrome

Source: LeetCode 234 — Palindrome Linked List

Problem: Check whether a singly linked list reads the same forwards and backwards.

Example

```
Input:  1 -> 2 -> 2 -> 1
Output: true
```

Approach: Find the middle (Q68), reverse the second half (Q66), then walk both halves together comparing values.

```
    static boolean isPalindromeList(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }

        ListNode secondHalf = reverseList(slow);   // reuse Q66
        ListNode firstHalf = head;
        while (secondHalf != null) {
            if (firstHalf.val != secondHalf.val) return false;
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
        }
        return true;
    }
```

Complexity: Time O(n), Space O(1)

Q73. Detect and Remove a Loop in a Linked List

Source: LeetCode 142 — Linked List Cycle II (LC asks you to find the start of the cycle; this version goes one step further and unlinks it entirely).

Problem: Detect a cycle in a linked list, and if one exists, remove it so the list becomes a normal, terminating list.

Example

```
Input:  1 -> 2 -> 3 -> 4 -> back to 2
Output: list becomes 1 -> 2 -> 3 -> 4 -> null
```

Approach: First use Floyd's algorithm (Q67) to detect the cycle and find the meeting point. Then reset one pointer to `head`, keep the other at the meeting point, and advance both one step at a time — they'll meet exactly at the **start** of the cycle. From there, walk to the last node before the cycle repeats and set its `next` to `null`.

> **Teaching note:** Why does resetting one pointer to `head` and stepping both by 1 find the cycle's start? It comes down to the math of where `slow` and `fast` first meet relative to the cycle's entry point — the distance from `head` to the cycle start turns out to exactly equal the distance from the meeting point back around to the cycle start. It's a beautiful (if non-obvious) result — most people just memorize the two-phase pattern: phase 1 detects a meeting point, phase 2 finds the entry.

```
    static void detectAndRemoveLoop(ListNode head) {
        ListNode slow = head, fast = head;
        boolean hasLoop = false;
        while (fast != null && fast.next != null) {
            slow = slow.next; fast = fast.next.next;
            if (slow == fast) { hasLoop = true; break; }
        }
        if (!hasLoop) return;

        slow = head;
        if (slow == fast) {
            // special case: loop starts at head itself
            while (fast.next != slow) fast = fast.next;
        } else {
            while (slow.next != fast.next) { slow = slow.next; fast = fast.next; }
        }
        fast.next = null;   // cut the loop
    }
```

Complexity: Time O(n), Space O(1)

Q74. Add Two Numbers Represented as Linked Lists (Digits in Reverse Order)

Source: LeetCode 2 — Add Two Numbers

Problem: Two numbers are represented as linked lists with digits stored in reverse order (least significant digit first). Add them and return the sum as a linked list in the same format.

Example

```
Input:  l1 = 2 -> 4 -> 3 (represents 342), l2 = 5 -> 6 -> 4 (represents 465)
Output: 7 -> 0 -> 8 (represents 807)
```

Approach: Simulate elementary-school column addition — walk both lists together, add corresponding digits plus any carry, and create a new node for each digit of the result.

```java
        static ListNode addTwoNumbers(ListNode l1, ListNode l2) {
            ListNode dummy = new ListNode(0);
            ListNode curr = dummy;
            int carry = 0;
            while (l1 != null || l2 != null || carry != 0) {
                int sum = carry + (l1 != null ? l1.val : 0) + (l2 != null ? l2.val : 0);
                carry = sum / 10;
                curr.next = new ListNode(sum % 10);
                curr = curr.next;
                if (l1 != null) l1 = l1.next;
                if (l2 != null) l2 = l2.next;
            }
            return dummy.next;
        }
```

Complexity: Time O(max(n,m)), Space O(max(n,m))

Q75. Find the Intersection Point of Two Linked Lists

Source: LeetCode 160 — Intersection of Two Linked Lists

Problem: Two singly linked lists eventually merge into one (they intersect). Find the node where they start sharing the same tail.

Example

```
A: 4 -> 1 -> 8 -> 4 -> 5
B:      5 -> 6 -> 1 -> 8 -> 4 -> 5    (shares 8 -> 4 -> 5 with A)
Output: node with value 8
```

Approach — pointer swap trick: Walk pointer `a` down list A then list B; walk pointer `b` down list B then list A. Both pointers travel the exact same total distance (`lenA + lenB`) before potentially meeting, which automatically cancels out any length difference between the two lists — so they arrive at the intersection point (or both reach `null`) at the same time.

> **Teaching note:** This is a genuinely elegant trick worth internalizing: if list A has length `a` and list B has length `b`, then pointer `a` travels `a + b` total steps by switching lists once, and so does pointer `b`. That means by the time either pointer reaches the intersection node, both have covered identical total distance — so they're guaranteed to land on it simultaneously, no length calculation needed up front.

```java
        static ListNode getIntersectionNode(ListNode headA, ListNode headB) {
            if (headA == null || headB == null) return null;
            ListNode a = headA, b = headB;
            while (a != b) {
                a = (a == null) ? headB : a.next;    // switch to the other list once exhausted
                b = (b == null) ? headA : b.next;
            }
            return a;    // either the intersection node, or null if no intersection
        }
```

Complexity: Time O(n+m), Space O(1)

Section 6: Stack & Queue (Q76–Q83)

Q76. Implement a Stack Using an Array

Source: Not on LeetCode — a fundamental data-structure design exercise.

Problem: Implement `push`, `pop`, `peek`, and `isEmpty` for a stack backed by a fixed-size array.

Approach: Track a `top` index, starting at -1 (empty). `push` increments then writes; `pop` reads then decrements.

```
class ArrayStack {
    int[] arr;
    int top;
    ArrayStack(int capacity) { arr = new int[capacity]; top = -1; }
    void push(int x) { arr[++top] = x; }
    int pop() { return arr[top--]; }
    int peek() { return arr[top]; }
    boolean isEmpty() { return top == -1; }
}
```

Complexity: Time O(1) per operation, Space O(n)

Q77. Check for Balanced Parentheses Using a Stack

Source: LeetCode 20 — Valid Parentheses — see Q40 for the full walkthrough and solution; it's the same problem, just grouped under "Stack" here since that's the pattern interviewers usually want you to name.

Q78. Implement a Queue Using Two Stacks

Source: LeetCode 232 — Implement Queue using Stacks

Problem: Implement FIFO queue behavior using only two LIFO stacks.

Example

```
enqueue(1), enqueue(2), dequeue() -> 1, enqueue(3), dequeue() -> 2
```

Approach: `enqueue` just pushes onto `s1`. For `dequeue`, if `s2` is empty, pour everything from `s1` into `s2` first — this reverses the order, putting the oldest element on top of `s2`, ready to pop.

> **Teaching note:** The "only refill s2 when it's empty" rule is what keeps this efficient. Once elements are poured into s2, they stay in the correct dequeue order for as long as needed — you don't need to re-pour on every single dequeue call, only when s2 runs dry. That's why it's O(1) *amortized* even though a single pour operation can be O(n).

```
class QueueUsingStacks {
    Deque<Integer> s1 = new ArrayDeque<>(), s2 = new ArrayDeque<>();
    void enqueue(int x) { s1.push(x); }
    int dequeue() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) s2.push(s1.pop());   // reverse order into s2
        }
        return s2.pop();
    }
}
```

Complexity: Time O(1) amortized per operation, Space O(n)

Q79. Find the Next Greater Element for Each Element in an Array

Source: LeetCode 496 — Next Greater Element I

Problem: For each element, find the first element to its right that's strictly greater; use -1 if none exists.

Example

```
Input:  arr = [4, 5, 2, 25]
Output: [5, 25, 25, -1]
```

Approach — monotonic decreasing stack of indices: Walk left to right. Before pushing the current index, pop off (and resolve) every index on the stack whose value is smaller than the current element — the current element *is* their next greater element.

> **Teaching note:** The stack always holds indices of elements that are still "waiting" for a bigger value to their right, in decreasing order of value from bottom to top. When a bigger element finally shows up, it resolves every smaller waiting element at once (that's the `while` loop) before joining the stack itself. This monotonic-stack pattern shows up constantly in interview problems — "next greater/smaller" of any kind is almost always this exact template.

```java
static int[] nextGreaterElement(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();   // stores indices, values decreasing bottom to top
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
            result[stack.pop()] = arr[i];   // arr[i] is the next greater element for this index
        }
        stack.push(i);
    }
    return result;
}
```

Complexity: Time O(n), Space O(n)

Q80. Evaluate a Postfix (Reverse Polish) Expression Using a Stack

Source: LeetCode 150 — Evaluate Reverse Polish Notation

Problem: Evaluate a postfix expression like `["2","1","+","3","*"]` (= `(2+1)*3` = `9`).

Example

```
Input:  tokens = ["2", "1", "+", "3", "*"]
Output: 9
```

Approach: Push numbers onto the stack. When an operator appears, pop the top two operands (note the order: the second-popped is the left operand), apply the operator, and push the result back.

```java
static int evalPostfix(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String token : tokens) {
        if (token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/")) {
            int b = stack.pop(), a = stack.pop();   // b popped first = right operand
            switch (token) {
                case "+": stack.push(a + b); break;
                case "-": stack.push(a - b); break;
                case "*": stack.push(a * b); break;
                case "/": stack.push(a / b); break;
            }
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}
```

Complexity: Time O(n), Space O(n)

Q81. Implement a Stack That Supports getMinimum() in O(1)

Source: LeetCode 155 — Min Stack

Problem: Design a stack that supports `push`, `pop`, `top`, and retrieving the minimum element — all in O(1).

Approach: Maintain a second "shadow" stack alongside the main one, where each position stores the minimum value *at that point in time*. Pushing to `minStack` always pushes `min(newValue, currentMin)`, so both stacks stay perfectly in sync and popping either one automatically keeps the minimum accurate.

```
class MinStack {
    Deque<Integer> stack = new ArrayDeque<>();
    Deque<Integer> minStack = new ArrayDeque<>();

    void push(int x) {
        stack.push(x);
        minStack.push(minStack.isEmpty() ? x : Math.min(x, minStack.peek()));
    }
    void pop() { stack.pop(); minStack.pop(); }
    int top() { return stack.peek(); }
    int getMin() { return minStack.peek(); }
}
```

Complexity: Time O(1) per operation, Space O(n)

Q82. Sort a Stack Using Recursion (No Other Data Structure)

Source: Not on LeetCode — classic GfG recursion exercise.

Problem: Sort a stack's elements using only recursion (the call stack is your only extra "storage").

Approach: Recursively pop everything off the stack down to empty, then as the recursion unwinds, insert each popped value back into its correct sorted position using a helper that itself recurses to find that position.

```
static void sortStack(Deque<Integer> stack) {
    if (!stack.isEmpty()) {
        int top = stack.pop();
        sortStack(stack);              // sort everything below first
        insertSorted(stack, top);      // then insert this value in the right spot
    }
}
static void insertSorted(Deque<Integer> stack, int val) {
    if (stack.isEmpty() || stack.peek() <= val) {
        stack.push(val);
        return;
    }
    int top = stack.pop();
    insertSorted(stack, val);   // keep looking deeper for the right spot
    stack.push(top);            // restore this element on top
}
```

Complexity: Time O(n²), Space O(n) recursion stack

Q83. Implement a Circular Queue With a Fixed-Size Array

Source: LeetCode 622 — Design Circular Queue

Problem: Implement a fixed-capacity queue that wraps indices around instead of shifting elements.

Approach: Track `front`, `rear`, and `count`. Use modulo arithmetic so `rear` / `front` wrap back to 0 once they hit the array's end.

```
class CircularQueue {
    int[] arr;
    int front, rear, count, capacity;

    CircularQueue(int size) {
        arr = new int[size];
        capacity = size;
        front = 0; rear = -1; count = 0;
    }
    boolean enqueue(int x) {
        if (count == capacity) return false;
        rear = (rear + 1) % capacity;   // wrap around
        arr[rear] = x;
        count++;
        return true;
    }
    int dequeue() {
        if (count == 0) return -1;
        int val = arr[front];
        front = (front + 1) % capacity;
        count--;
        return val;
    }
}
```

Complexity: Time O(1) per operation, Space O(n)

Section 7: HashMap / HashSet (Q84–Q90)

Q84. Two Sum — Return Indices of Two Numbers That Add Up to a Target

Source: LeetCode 1 — Two Sum — arguably the single most well-known interview question, and a great one to have completely automatic.

Problem: Return the indices of the two numbers that add up to a target value.

Example

```
Input:  arr = [2, 7, 11, 15], target = 9
Output: [0, 1]
Explanation: arr[0] + arr[1] = 2 + 7 = 9.
```

Approach: Scan once, storing each value's index in a HashMap as you go. Before storing the current value, check whether its complement (`target - arr[i]`) is already in the map — if so, you've found your pair immediately.

```java
static int[] twoSum(int[] arr, int target) {
    Map<Integer, Integer> map = new HashMap<>();   // value -> index
    for (int i = 0; i < arr.length; i++) {
        int need = target - arr[i];
        if (map.containsKey(need)) return new int[]{map.get(need), i};
        map.put(arr[i], i);
    }
    return new int[]{-1, -1};
}
```

Complexity: Time O(n), Space O(n)

Q85. Find the First Repeating Element in an Array

Source: GfG classic *First Repeating Element* — not its own numbered LeetCode problem.

Problem: Find the element that occurs more than once and whose **first occurrence** is earliest among all repeating elements.

Example

```
Input:  arr = [10, 5, 3, 4, 3, 5, 6]
Output: 5
Explanation: Both 3 and 5 repeat, but 5's first occurrence (index 1) is earlier than 3's (index 2).
```

Approach: Scan from the **right**, adding each value to a HashSet. If a value is already in the set, it repeats later in the array — keep overwriting `result` as you go, so by the time you finish scanning right-to-left, `result` holds the repeating value with the *earliest* first occurrence.

```java
static int firstRepeating(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    int result = -1;
    for (int i = arr.length - 1; i >= 0; i--) {
        if (seen.contains(arr[i])) result = arr[i];   // keep overwriting — leftmost wins
        seen.add(arr[i]);
    }
    return result;
}
```

Complexity: Time O(n), Space O(n)

Q86. Group Anagrams Together From an Array of Strings

Source: LeetCode 49 — Group Anagrams

Problem: Group strings that are anagrams of each other.

Example

```
Input:  strs = ["eat","tea","tan","ate","nat","bat"]
Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

Approach: Anagrams share the same characters, so sorting each string's letters produces an identical "key" for every anagram in the group. Use that sorted string as a HashMap key and bucket the originals under it.

```
        static List<List<String>> groupAnagrams(String[] strs) {
            Map<String, List<String>> map = new HashMap<>();
            for (String s : strs) {
                char[] c = s.toCharArray();
                Arrays.sort(c);
                String key = new String(c);   // anagrams all produce the same sorted key
                map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
            }
            return new ArrayList<>(map.values());
        }
```

Complexity: Time O(n × k log k) where k is max string length, Space O(n × k)

Q87. Find All Pairs With a Given Absolute Difference k

Source: LeetCode 532 — K-diff Pairs in an Array

Problem: Find all pairs `(x, x+k)` present in the array.

Example

```
Input:  arr = [1, 5, 3, 4, 2], k = 1
Output: (1,2), (3,4), (4,5)   -- e.g., printed in scan order
```

Approach: Put every value in a HashSet for O(1) lookups. For each value `x`, just check whether `x + k` also exists.

```
        static List<int[]> pairsWithDiff(int[] arr, int k) {
            Set<Integer> set = new HashSet<>();
            for (int x : arr) set.add(x);
            List<int[]> result = new ArrayList<>();
            for (int x : arr) {
                if (set.contains(x + k)) result.add(new int[]{x, x + k});
            }
            return result;
        }
```

Complexity: Time O(n), Space O(n)

Q88. Check if the Array Contains Duplicates Within Distance k of Each Other

Source: LeetCode 219 — Contains Duplicate II

Problem: Check whether there exist two equal elements whose indices are at most k apart.

Example

```
Input:  arr = [1, 2, 3, 1], k = 3
Output: true
```

Approach: Keep a HashMap of each value's most recent index. If the value reappears and the index gap is ≤ k, return true immediately; otherwise update the stored index and keep scanning.

```
        static boolean containsNearbyDuplicate(int[] arr, int k) {
            Map<Integer, Integer> lastIndex = new HashMap<>();
            for (int i = 0; i < arr.length; i++) {
                if (lastIndex.containsKey(arr[i]) && i - lastIndex.get(arr[i]) <= k) return true;
                lastIndex.put(arr[i], i);
            }
            return false;
        }
```

Complexity: Time O(n), Space O(n)

Q89. Find the Length of the Longest Consecutive Sequence in an Unsorted Array

Source: LeetCode 128 — Longest Consecutive Sequence

Problem: Find the length of the longest run of consecutive integers (e.g., `4,5,6,7`), in O(n) — sorting first would cost O(n log n), so this needs a smarter approach.

Example

```
Input:  arr = [100, 4, 200, 1, 3, 2]
Output: 4
Explanation: The sequence 1, 2, 3, 4 has length 4.
```

Approach: Put everything in a HashSet. Only *start* counting a sequence from a number that has no predecessor in the set (i.e., `x - 1` is not present) — that guarantees each number is only ever the start of at most one sequence, which is what keeps this O(n) instead of O(n²).

> **Teaching note:** The `if (!set.contains(x - 1))` check is the whole trick. Without it, you'd re-count the same sequence multiple times (once from every number inside it), turning an O(n) algorithm into something much slower. By only ever starting from a true "sequence beginning," every number in the array gets visited by the inner `while` loop at most once across the *entire* run of the algorithm — so total work stays linear.

```java
static int longestConsecutive(int[] arr) {
    Set<Integer> set = new HashSet<>();
    for (int x : arr) set.add(x);

    int longest = 0;
    for (int x : set) {
        if (!set.contains(x - 1)) {   // x is the start of a sequence
            int length = 1;
            while (set.contains(x + length)) length++;
            longest = Math.max(longest, length);
        }
    }
    return longest;
}
```

Complexity: Time O(n), Space O(n)

Q90. Count Distinct Elements in Every Window of Size k

Source: GfG classic *Count distinct elements in every window of size k* — not its own numbered LeetCode problem, but a natural sliding-window/HashMap combination worth knowing.

Problem: For every contiguous window of size k, count how many distinct elements it contains.

Example

```
Input:  arr = [1, 2, 1, 3, 4, 2, 3], k = 4
Output: [3, 4, 4, 3]
```

Approach — sliding window with a frequency map: As the window slides, add the newly entered element's count and remove the element that just fell out of the window (deleting its key entirely once its count hits zero, so `map.size()` always reflects the true count of distinct elements currently in the window).

```java
static List<Integer> countDistinctInWindow(int[] arr, int k) {
    List<Integer> result = new ArrayList<>();
    Map<Integer, Integer> freq = new HashMap<>();
    for (int i = 0; i < arr.length; i++) {
        freq.put(arr[i], freq.getOrDefault(arr[i], 0) + 1);   // element enters the window
        if (i >= k) {
            int outVal = arr[i - k];
            freq.put(outVal, freq.get(outVal) - 1);
            if (freq.get(outVal) == 0) freq.remove(outVal);   // clean up so size() stays accurate
        }
        if (i >= k - 1) result.add(freq.size());
    }
    return result;
}
```

Complexity: Time O(n), Space O(k)

Section 8: Sorting & Searching (Q91–Q95)

Q91. Implement Bubble Sort

Source: Not on LeetCode as a standalone problem, but LeetCode 912 — Sort an Array is a good place to practice implementing any sorting algorithm from scratch (bubble sort will time out on LC's large inputs, but it's a must-know fundamental).

Problem: Sort an array using the classic bubble sort approach.

Example

```
Input:  arr = [5, 1, 4, 2, 8]
Output: [1, 2, 4, 5, 8]
```

Approach: Repeatedly walk the array, swapping any adjacent out-of-order pair. Each full pass "bubbles" the largest remaining value to its correct position at the end. A `swapped` flag lets you exit early once a pass makes no swaps (the array is already sorted).

```java
static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;   // already sorted, stop early
    }
}
```

Complexity: Time O(n²) worst case, O(n) best case (already sorted), Space O(1)

Q92. Implement Merge Sort

Source: LeetCode 912 — Sort an Array is the standard place to implement and test this.

Problem: Sort an array using the divide-and-conquer merge sort algorithm.

Example

```
Input:  arr = [38, 27, 43, 3, 9, 82, 10]
Output: [3, 9, 10, 27, 38, 43, 82]
```

Approach: Recursively split the array in half until each piece has one element (trivially sorted), then merge sorted halves back together (the same "merge" logic as Q16).

```java
static void mergeSort(int[] arr, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    merge(arr, l, mid, r);
}
static void merge(int[] arr, int l, int mid, int r) {
    int[] temp = new int[r - l + 1];
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r) temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];
    System.arraycopy(temp, 0, arr, l, temp.length);
}
```

Complexity: Time O(n log n), Space O(n)

Q93. Implement Quick Sort

Source: LeetCode 912 — Sort an Array

Problem: Sort an array using the quick sort algorithm.

Example

```
Input:  arr = [10, 7, 8, 9, 1, 5]
Output: [1, 5, 7, 8, 9, 10]
```

Approach: Pick a pivot (here, the last element), partition the array so everything smaller ends up to its left and everything bigger to its right, then recursively sort each side.

```
        static void quickSort(int[] arr, int low, int high) {
            if (low < high) {
                int pi = partition(arr, low, high);
                quickSort(arr, low, pi - 1);
                quickSort(arr, pi + 1, high);
            }
        }
        static int partition(int[] arr, int low, int high) {
            int pivot = arr[high];
            int i = low - 1;                        // boundary of "smaller than pivot" region
            for (int j = low; j < high; j++) {
                if (arr[j] < pivot) {
                    i++;
                    int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
                }
            }
            int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;   // place pivot in its final spot
            return i + 1;
        }
```

Complexity: Time O(n log n) average, O(n²) worst case (already-sorted input with a bad pivot choice), Space O(log n)

Q94. Find the Kth Smallest Element Using QuickSelect

Source: GfG classic *Kth smallest element*; the mirror-image variant of LeetCode 215 — Kth Largest Element in an Array, solvable with the same technique.

Problem: Find the kth smallest element without fully sorting the array.

Example

```
Input:  arr = [7, 10, 4, 3, 20, 15], k = 3
Output: 7
```

Approach: Reuse quick sort's `partition` step, but only recurse into the *one* side that could actually contain the kth position — never both sides. On average this cuts the search space roughly in half each time instead of fully sorting.

> **Teaching note:** This is the same idea as binary search applied to partitioning instead of comparison. `partition` tells you exactly where the pivot landed (`pi`) relative to the sorted order. If `pi == k` , you're done. If `pi < k` , the answer must be somewhere to the right, so only recurse right — and vice versa. Throwing away the unneeded half is what brings the average time down from O(n log n) (full sort) to O(n).

```
        static int quickSelect(int[] arr, int low, int high, int k) {
            if (low == high) return arr[low];
            int pi = partition(arr, low, high);   // reuses Q93's partition
            if (pi == k) return arr[pi];
            else if (pi < k) return quickSelect(arr, pi + 1, high, k);
            else return quickSelect(arr, low, pi - 1, k);
        }
        // call: quickSelect(arr, 0, arr.length - 1, k - 1) for the kth smallest (1-indexed input)
```

Complexity: Time O(n) average, O(n²) worst case, Space O(log n)

Q95. Count Inversions in an Array Using Merge Sort

Source: GfG classic *Count Inversions*; LeetCode 493 — Reverse Pairs is a harder numbered variant built on the same merge-sort-counting idea.

Problem: Count the number of pairs `(i, j)` where `i < j` but `arr[i] > arr[j]` — a measure of how "unsorted" the array is.

Example

```
Input:  arr = [2, 4, 1, 3, 5]
Output: 3
Explanation: The inversions are (2,1), (4,1), (4,3).
```

Approach: Piggyback on merge sort's merge step. Whenever an element from the **right** half gets placed before the left half is exhausted, every remaining element in the left half forms an inversion with it (because the left half is sorted, they're all bigger).

> **Teaching note:** This is a great example of "free" extra information hiding inside an algorithm you already know. Merge sort already compares elements from the left and right halves during merging — counting inversions just means *noticing* that every time you take from the right half early, you can add `(mid - i + 1)` inversions in one shot (that's how many left-half elements are still waiting and are therefore all bigger than the value you just took).

```java
static long countInversions(int[] arr, int l, int r) {
    long count = 0;
    if (l < r) {
        int mid = l + (r - l) / 2;
        count += countInversions(arr, l, mid);
        count += countInversions(arr, mid + 1, r);
        count += mergeAndCount(arr, l, mid, r);
    }
    return count;
}
static long mergeAndCount(int[] arr, int l, int mid, int r) {
    int[] temp = new int[r - l + 1];
    int i = l, j = mid + 1, k = 0;
    long invCount = 0;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
            invCount += (mid - i + 1);   // all remaining left-half elements form inversions with arr[j]
        }
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];
    System.arraycopy(temp, 0, arr, l, temp.length);
    return invCount;
}
```

Complexity: Time O(n log n), Space O(n)

Section 9: Recursion & Basic Math (Q96–Q100)

Q96. Find the Factorial of a Number (Recursive)

Source: Not on LeetCode as a standalone problem; LeetCode 172 — Factorial Trailing Zeroes is a related, trickier factorial question worth knowing too.

Problem: Compute `n!` recursively.

Example

```
Input:  n = 5
Output: 120
```

Approach: Base case: `factorial(1) = 1`. Recursive case: `n × factorial(n-1)`.

```java
static long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Complexity: Time O(n), Space O(n) recursion stack

Q97. Generate the Fibonacci Series up to n Terms (Iterative)

Source: LeetCode 509 — Fibonacci Number

Problem: Generate the first n terms of the Fibonacci sequence.

Example

```
Input:  n = 6
Output: [0, 1, 1, 2, 3, 5]
```

Approach: Build the sequence iteratively, each term the sum of the previous two.

> **Teaching note:** Plain recursive Fibonacci (`fib(n) = fib(n-1) + fib(n-2)`) looks elegant but is $O(2^n)$ — it recomputes the same smaller values over and over. In a 45-minute timed round that can literally time out for even moderately large n. The iterative version below (or memoized recursion) is what you actually want to write.

```java
static int[] fibonacci(int n) {
    int[] fib = new int[n];
    for (int i = 0; i < n; i++) {
        if (i == 0) fib[i] = 0;
        else if (i == 1) fib[i] = 1;
        else fib[i] = fib[i - 1] + fib[i - 2];
    }
    return fib;
}
```

Complexity: Time O(n), Space O(n)

Q98. Check if a Number is Prime

Source: Not on LeetCode as a single-number check; LeetCode 204 — Count Primes is the related “find all primes up to n” version, solved with the Sieve of Eratosthenes instead.

Problem: Determine whether a number is prime.

Example

```
Input:  n = 29
Output: true
```

Approach: You only need to test divisors up to $\sqrt{n}$ — if n has a factor larger than its square root, it must also have a matching factor smaller than the square root, so that smaller one would already have been caught.

```java
    static boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; (long) i * i <= n; i += 2) {    // only check odd divisors up to sqrt(n)
            if (n % i == 0) return false;
        }
        return true;
    }
```

Complexity: Time O(sqrt(n)), Space O(1)

Q99. Check if a Number is an Armstrong Number

Source: Not on LeetCode — classic HackerRank/GfG number-theory problem. This exact "sum of powers of digits" style question has been directly reported in Accenture's coding round.

Problem: A number is an Armstrong number if the sum of each digit raised to the power of the total digit count equals the number itself (e.g., 153 = $1^3 + 5^3 + 3^3$).

Example

```
Input:  num = 153
Output: true
Explanation: 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.
```

Approach: Count the digits first, then peel off digits one at a time (via `% 10` and `/ 10`), raising each to the digit-count power and summing.

```java
    static boolean isArmstrong(int num) {
        int original = num, numDigits = String.valueOf(num).length();
        double sum = 0;
        while (num > 0) {
            int digit = num % 10;
            sum += Math.pow(digit, numDigits);
            num /= 10;
        }
        return (int) sum == original;
    }
```

Complexity: Time O(log n) — proportional to the number of digits, Space O(1)

Q100. Find the GCD and LCM of Two Numbers

Source: Not on LeetCode as a single combined problem; LeetCode 1979 — Find Greatest Common Divisor of an Array uses the same Euclidean GCD idea.

Problem: Find both the Greatest Common Divisor and Least Common Multiple of two numbers.

Example

```
Input:  a = 12, b = 18
Output: gcd = 6, lcm = 36
```

Approach: Euclid's algorithm computes GCD recursively (`gcd(a, b) = gcd(b, a % b)`, base case `gcd(a, 0) = a`). Once you have the GCD, LCM falls out for free from the identity `LCM(a,b) × GCD(a,b) = a × b`.

```java
    static int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }
    static long lcm(int a, int b) {
        return ((long) a * b) / gcd(a, b);
    }
```

Complexity: Time O(log(min(a,b))), Space O(log(min(a,b))) recursion stack

Final Prep Tips for the Accenture 2026 Coding Round

1. **Practice speed, not just correctness** — you get 2 questions in 45 minutes, so aim to solve an easy Array/String problem in under 10–12 minutes to leave enough time for the medium one.
2. **Master Arrays and Strings first** — real reported questions include “leader elements,” longest substring, array/matrix manipulation, and digit-sum-based number checks (Armstrong/round numbers) almost every cycle.
3. **Write clean, compiling Java** — scoring is test-case based with partial credit, so a working brute-force solution that passes most test cases beats an elegant one that doesn’t compile.
4. **Watch edge cases**: empty arrays/strings, single-element input, all-duplicate arrays, negative numbers — these are exactly what hidden test cases tend to target.
5. Binary Search, Linked List, and Stack/Queue questions are more likely in the **technical interview** round (especially for Advanced ASE) than in the timed coding test — but knowing them cold also makes you faster at spotting the right approach under time pressure in the coding round itself.
6. **Revisit the Teaching notes** in this guide before your test — they’re concentrated on exactly the patterns (Kadane’s, Floyd’s cycle detection, Dutch National Flag, sliding windows, monotonic stacks, the merge-sort family) that come up again and again across *different-looking* problems once you know how to spot them.